

AI-Setup & Arbeit mit Agenten

Alain Horner · 16. September 2026

Dieses Dokument gibt einen Überblick über die Arbeit mit AI [Agenten](#) im SEB Server Projekt. Es orientiert sich stark an der Arbeit von [Matt Pocock](#). Momentan arbeiten wir mit [Claude Code](#).

Grundprinzipien

- **Feedback-Loops sind alles.** Agenten werden massiv besser, wenn sie ihre Arbeit selbst verifizieren können ([Automated Checks](#)): Playwright (oder andere automatisierte Tests), Typecheck, ESLint, Prettier, Browser-Zugriff. Jede Investition in deterministische Tools zahlt sich doppelt aus, weil sie nicht nur uns, sondern auch die AI besser macht.
- **Die AI imitiert, was sie sieht.** Schlechte Gewohnheiten in der Codebase beeinflussen den generierten Code direkt. Aufräumen ist keine Kosmetik, sondern verbessert auch den Output der AI.
- **Kontext ist eine knappe Ressource.** Das [Kontext](#)-Fenster des Agenten ist begrenzt, und die Qualität sinkt schon, bevor es voll ist. Deshalb: eine Aufgabe pro Session; zwischen einzelnen Aufgaben `/clear` benutzen; relevantes Wissen in kleinen, gezielten Files ablegen, statt es in langen Chat-Verläufen mitzuschleppen. Alles, was permanent im Kontext landet (z.B. `CLAUDE.md`, MCP-Server), muss seinen Platz verdienen.
- **Niemals Code committen, den wir nicht verstehen.** Generierten Code so lange reviewen und tweakern ([Human Review](#)), bis alles so ist, wie wir es selbst geschrieben hätten. Das ist die Grenze zu [Vibe-Coding](#).

Arbeitsmodus

- **Workflow:** [aihero.dev/skills](#) erklärt den Workflow, nach dem wir arbeiten (inkl. Doku und Erklärvideos). Kurzfassung: Idee per [Grilling](#) klar definieren / Gegenfragen provozieren → [Spec](#) → [Tickets](#) → Umsetzung. Dazu gibt es ein gutes [Intro-Video](#).
- **Der Mensch bleibt dabei ([Human-in-the-Loop](#)):** Der Mensch plant zusammen mit dem Agenten, der Code wird reviewed, der Agent wird auf die richtige Bahn zurückgebracht, wenn er sich verirrt. Ob man den Code auf GitHub oder lokal reviewed, ist egal. Hauptsache, man tut es. Damit stellt man nicht nur die Codequalität sicher, sondern auch, dass man nicht den Überblick verliert.

- **Verifikation während der Entwicklung:** Der Agent muss die vorhandenen Verifikationstools verwenden (siehe Feedback-Loops). Mit Playwright-MCP kann der Agent die Änderung selbständig im Browser gegen den laufenden Dev-Server testen, v.a. dort wo automatisierte Tests (noch) fehlen. Sobald wir eine saubere automatisierte Testabdeckung haben, verliert dies an Bedeutung (automatische Tests sind billiger und schneller). Während der Entwicklung bleibt es für Claude aber ein wichtiges Tool.

Setup & Werkzeuge

- **Sandbox:** Aus Sicherheitsgründen lassen wir die Agenten grundsätzlich in [Docker AI Sandboxes](#) laufen. Alternativ kann z.B. auch [Claude Code Devcontainer](#) verwendet werden. Wichtig ist, dass der File- und Internetzugriff der Agenten eingeschränkt wird.
- **MCP-Server** (eingetragen in `.mcp.json`):
 - **Playwright MCP:** Gibt den Agenten Zugriff auf einen (sandboxed) Browser, in dem sie die Applikation "als User" testen können.
 - **Vuetify MCP:** Vuetify-API-Doku und Guides für den Agenten.
- **Skills:** Die [Skills von Matt Pocock](#) sind installiert und unter `.claude/skills/` im Repo eingetragten. Sie sind das Rückgrat des oben beschriebenen Workflows. Wenn man nicht weiss, was man als nächstes machen soll: einfach den `/ask-matt`-Skill benutzen.
- **Autoresearch-Skill:** Hilfreich, wenn man auf ein messbares Optimierungsziel hinarbeitet (z.B. Download-Geschwindigkeit, Bundle-Grösse des Client-Builds, ...).

Wissensablage im Repo

- `.claude/CLAUDE.md` : Einstiegsfile für Claude. Dieses sollte möglichst klein gehalten werden, um den [Kontext](#) nicht unnötig zu füllen.
- `.claude/rules/client.md` : Konkrete Coding-Regeln, als [Rules file](#) (CLAUDE.md-Instruktionen, limitiert auf den `client` Ordner). Unbedingt up2date halten: So schreibt Claude den Code von Anfang an so, wie wir wollen. Das File dient gleichzeitig uns Menschen als Nachschlagewerk.
- `.scratch/` : Die technischen Tickets und die Resultate aus Grilling-Sessions. Sie helfen den Agenten, den [Kontext](#) und vergangene Entscheidungen zu verstehen. Wird es irgendwann zu viel, kann man alte Ordner einfach löschen. Es gibt auch die Möglichkeit, die technischen Tickets direkt im [Jira](#) abzulegen.
- `client/CONTEXT.md` (+ `CONTEXT-MAP.md` im Root): Glossar mit den Begriffsdefinitionen. Dieses wird von Grilling-Sessions aktiv gepflegt.

Empfehlungen

1. **Automatisierte Testabdeckung aufbauen.** Stärkt den Feedback-Loop der Agenten und reduziert manuelle Arbeit. Siehe Empfehlung 1 in [infos-projekt.md](#) zu konkreten Strategie.

2. **Einfach mal ausprobieren.** Solange man in der Sandbox ist, kann man kaum Schaden anrichten, der sich nicht mit Git wieder rückgängig machen lässt.
3. **Verstehen und am Ball bleiben.** Die Videos und Posts auf [AI-Hero](#) sind Gold wert. Es gibt auch Kurse.
4. `.claude/rules/client.md` **pflegen.** Jedes Mal, wenn man im Code-Review denselben Fehler zweimal korrigiert, gehört eine Regel dazu ins File.
5. **Codebase konsequent sauber halten.** Siehe Grundprinzipien: jede Altlast, die wir loswerden, verbessert den Output aller zukünftigen Sessions.
6. **Sich nicht verrückt machen lassen.** [AI-Burnout](#) verhindern.